From Entropy to Epiplexity

Rethinking Information for Computationally Bounded Intelligence

Marc Finzi, Shikai Qiu, Yiding Jiang, Pavel Izmailov, J. Zico Kolter, Andrew Gordon Wilson

Carnegie Mellon University, New York University

arXiv:2601.03220, version 2, 16 March 2026

Speaker: Alex Kravatsky

Three apparent paradoxes of information

1. Deterministic computation cannot increase information

Yet self-play and synthetic data can improve a model's capabilities.

2. Total information does not depend on prediction order

Yet language-model loss can differ between forward and reverse text.

3. Likelihood modelling targets the data distribution

Yet prediction can require inference absent from the generating procedure.

Section 5, Finzi et al. (2026). These are apparent tensions, not contradictions of the classical results.

Shannon entropy and Kolmogorov complexity

Shannon entropy: a distribution

$$p(x) = \Pr(X = x),$$
$$H(X) = \mathbb{E}[-\log_2 p(X)].$$

For discrete X, entropy is a lower bound on the expected length of any binary prefix code.

Uniform n-bit strings: $H(X) = n$.

A fixed string: $H(X) = 0$.

Kolmogorov complexity: a string

$$K(x) = \min_{p:\, \mathcal{U}(p)=x} |p|.$$

The shortest self-delimiting program that outputs x and halts on a fixed prefix-free universal Turing machine $\mathcal{U}$.

An alternating string is specified by its rule and length. Most uniform n-bit strings require at least $n - O(1)$ program bits.

$K(x)$ is uncomputable in general. Definition 1 and Section 2.1, Finzi et al. (2026).

Paradox 1: deterministic computation and new information

For discrete X and a fixed computable transformation f,

$$H(f(X)) \le H(X), \qquad K(f(x)) \le K(x) + K(f) + c.$$

The output has a description: specify the input and the program for f, then run it. These bounds do not restrict how long that computation takes.

Example: expanding a random seed

A pseudorandom generator transforms k seed bits into $n > k$ output bits. The output still has entropy at most k and a description of length at most $k + K(G) + O(\log_2 k)$. For a secure generator, no randomized test with runtime polynomial in k distinguishes the output from uniform data with non-negligible advantage.

AlphaZero learns a policy and a value function through self-play. Specifying its algorithm and random seed can take far fewer bits than storing the trained network, but reconstructing that network requires running the training procedure.

Negligible advantage: eventually smaller than k^{-m} for every fixed $m > 0$. Section 5.1 and Theorem 12, Finzi et al. (2026); the theorem uses polynomial-length advice independent of the sampled seed.

Paradox 2: equal information, different prediction costs

The chain rule gives the same joint entropy in either order:

$$H(X) + H(Y \mid X) = H(X, Y) = H(Y) + H(X \mid Y).$$

Each order describes the same pair. The identity does not bound the cost of evaluating or sampling from the conditional distributions.

Example: a bijection computable in polynomial time

Let X be uniform on n-bit strings and $Y = f(X)$. Since f is a bijection, Y is also uniform and both conditional entropies are zero.

- **Input first:** encode X in n bits, then compute $Y = f(X)$.
- **Output first:** encode Y in n bits, then recover $X = f^{-1}(Y)$.

One-wayness means that no randomized algorithm with runtime polynomial in n recovers X from $f(X)$ with non-negligible probability over uniform X.

Prediction order can therefore affect what a model can compute. The paper studies this with cellular-automaton pairs and reordered chess data.

Section 5.2 and Theorem 13, Finzi et al. (2026). The theorem also allows polynomial-length advice independent of the sampled input; one-wayness is an assumption.

Paradox 3: can prediction require more than generation?

Likelihood training minimizes

$$L_Q(P) = \mathbb{E}_{Y \sim Q}[-\log_2 P(Y)].$$

When the model class contains the data distribution Q, the minimum is attained at $P = Q$.

Does matching the distribution require only reproducing the generator's computations? Consider a mystery story:

The author generates the story

Choose the culprit, then write consistent clues. The culprit is already known.

The model predicts the reveal

Given only the clues, assign probabilities to possible culprits before the reveal.

Prediction requires inference from clues to culprit; generation does not. The paper calls this **induction**: inference about unobserved variables. Under a time bound, it may require a different algorithm from the generator's.

Section 5.3 and the mystery-story example in Section 5.3.1, Finzi et al. (2026).

Time-bounded probabilistic models

Fix a prefix-free universal Turing machine $\mathcal{U}$ and a time bound $T(n)$. An admissible program P specifies a distribution P on $\{0,1\}^n$ and supports:

- **Evaluation:** given x, return $P(x)$ within $T(n)$ steps.
- **Sampling:** given independent random bits, generate $x \sim P$ within $T(n)$ steps.

Both modes use the same distribution P, with $\sum_x P(x) = 1$.

$\mathcal{P}_T$ is the set of admissible programs; $|\mathrm{P}|$ is the program length in bits.

Definition 7, Finzi et al. (2026). The formal bound is non-decreasing and time-constructible.

Epiplexity: the two-part description

For X on $\{0,1\}^n$, choose an admissible model minimizing the expected two-part description length:

$$\mathrm{P}^\star \in \arg\min_{\mathrm{P}\in\mathcal{P}_T} \big\{ |\mathrm{P}| + \mathbb{E}[-\log_2 P(X)] \big\}.$$

Among minimizers, select one with the shortest program.

Epiplexity: model description

$$S_T(X) := |\mathrm{P}^\star|.$$

Length of the selected model's program.

Time-bounded entropy: data description

$$H_T(X) := \mathbb{E}[-\log_2 P^\star(X)].$$

Expected data-code length using that model.

$$\mathrm{MDL}_T(X) = S_T(X) + H_T(X).$$

Definition 8, Finzi et al. (2026).

Examples: random bits and a repeated pattern

Assume the length n is specified and the time bound permits the models below.

Uniform bits: $X \sim U_n$

A constant-length program assigns $P(x) = 2^{-n}$ to every n-bit string.

$$\mathrm{MDL}_T(X) \le n + c, \qquad H_T(X) \ge H(X) = n.$$

Consequently $S_T(X) \le c$: a large data-code length can accompany a small model description.

Alternating strings: two equally likely outcomes

$X = 0101\ldots$ or $1010\ldots$, each with probability $\frac{1}{2}$. A constant-length program generates either pattern in linear time.

$$S_T(X) = O(1), \qquad H_T(X) = O(1).$$

Here both the model description and the residual data code are small.

Cryptographic consequences of the time bound

- **Pseudorandom generator.** G maps k random seed bits to $n > k$ output bits. $H(G(U_k)) \leq k$, yet for a secure generator $H_{\mathrm{Poly}}(G(U_k)) \approx n$ and $S_{\mathrm{Poly}}(G(U_k)) = O(1)$.
- **One-way function.** Computing $f(x)$ takes polynomial time. No randomized polynomial-time algorithm finds a preimage with non-negligible probability over uniformly sampled inputs. For a one-way *permutation*, the paper proves that the two factorization orders have different time-bounded entropies.

Theorems 9 and 13, Finzi et al. (2026). These results assume security against non-uniform probabilistic polynomial-time algorithms.

Conditional epiplexity: predicting labels from inputs

For supervised data (X, Y), the decoder already has the input X. The model evaluates $P(y \mid x)$ and samples $Y \mid X = x$ within the bound for each fixed x.

$$\mathrm{P}^{\star}_{Y|X} \in \operatorname*{arg\,min}_{\mathrm{P} \in \mathcal{P}^{X}_{T}} \{|\mathrm{P}| + \mathbb{E}_{X,Y}[-\log_2 P(Y \mid X)]\}\,.$$

$$S_T(Y \mid X) := |\mathrm{P}^{\star}_{Y|X}|, \qquad H_T(Y \mid X) := \mathbb{E}_{X,Y}[-\log_2 P^{\star}(Y \mid X)].$$

Example: image classification

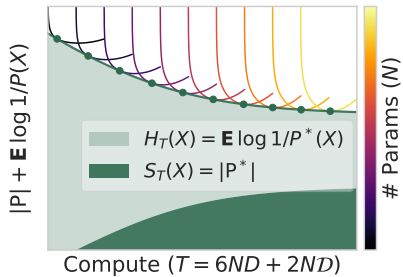
The model code describes the classifier. The data code describes the labels given the images; it does not encode the images.

These quantities are defined directly. In general, $H_T(Y \mid X) \neq H_T(X, Y) - H_T(X)$.

Definition 11, Finzi et al. (2026).

From programs to neural-network estimates

- Search over all programs is intractable, so fix an architecture and optimiser.
- Approximate dense-model accounting: $6ND + 2N\mathcal{D} \leq T$, where N counts parameters and $D, \mathcal{D}$ count training and evaluation tokens.
- Each run gives a model-part estimate plus a validation-data code. Select the shortest admissible two-part code; a lower hull smooths the empirical curve.
- Report its model and data parts as estimates $\widehat{S}_T$ and $\widehat{H}_T$.

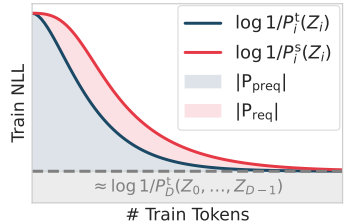


Schematic. Thin curves: training runs, coloured by N. Dots: the lower envelope. Dark green: S_T; light green: H_T. Source: Fig. 2b of Finzi et al. (2026).

Estimating the model part: prequential coding

- P_i has seen $Z_0, \ldots, Z_{i-1}$; P_0 is the initial model.
- Encode Z_i using its predicted probability, then update. The decoder repeats the update.
- The sequential data code sums these costs. Subtract the final-model data code to estimate the model part:

$$\widehat{K}_{\text{preq}} = \sum_{i<M} \left[-\log_2 P_i(Z_i) + \log_2 P_M(Z_i) \right].$$



Blue area: prequential model-part estimate. Red area: teacher–student loss gap used to visualise requential coding. Fig. 2a, Finzi et al. (2026).

- On an i.i.d. stream, the pre-update loss estimates loss on a fresh sample.
- The area is a heuristic model-part estimate; subtraction alone gives no model-only code.

An explicit model code: requential coding

- A teacher model trains on real data; a student model trains on teacher-distributed samples (Qiu et al., 2026).
- Both sides share the student and a random seed. Send the index of a student proposal selected to have the teacher's distribution.
- Relative entropy coding bounds the expected message length by a function of the teacher–student KL divergence:

$$\mathbb{E}[\ell_i \mid \mathcal{F}_i] \le \mathrm{KL}_i + 2\log_2(1 + \mathrm{KL}_i) + \kappa, \quad \kappa < 5.21,$$

Here $\mathrm{KL}_i = \mathbb{E}_{Z\sim P_i^{\mathrm{t}}}[\log_2(P_i^{\mathrm{t}}(Z)/P_i^{\mathrm{s}}(Z))]$ is the Kullback–Leibler divergence, and $\mathcal{F}_i$ is the prior history. $\sum_i \mathrm{KL}_i$ is the leading model-code estimate.

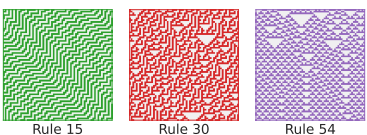
	Prequential	Requential
Model part	heuristic subtraction	explicit student reconstruction
Training	one network	teacher and student, 2–10× slower
Reported values	larger in the paper	smaller in the paper

The bound shown is from Qiu et al. (2026), Appendix A.1; the epiplexity paper used an earlier overhead expression.

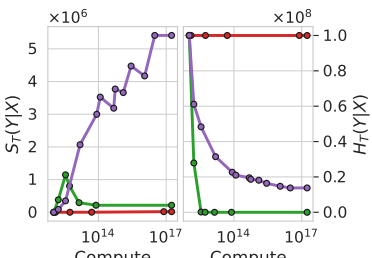
Cellular automata: simple dynamics, different model parts

An *elementary cellular automaton* (ECA) updates binary cells in parallel from their three-cell neighbourhoods (Wolfram, 2002).

Setting. 64 periodic cells; $Y = F_r^{48}(X)$; transformer sweep; requential code.



Rule	15	30	54
$S_T(Y \mid X)$, bits	2×10^5	≈ 0	5.4×10^6
$H_T(Y \mid X)$, bits	≈ 0	10^8	1.5×10^7

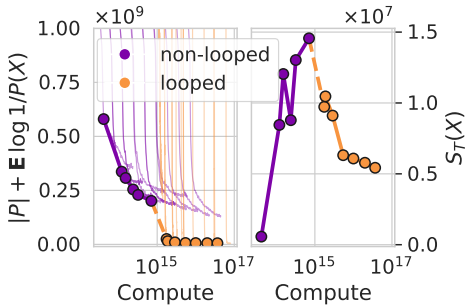


Left: rollouts, time downward. Right: S_T and H_T against compute in FLOPs. Table: our estimates from the plots at 10^{17} FLOPs. Source: Fig. 3 of Finzi et al. (2026).

Epiplexity need not increase with compute

Setting. ECA rule 54, 64 updates. The non-looped model predicts the final state; the looped model also predicts intermediate states.

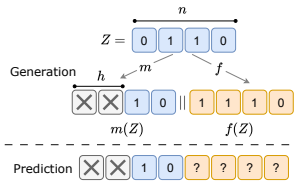
- Below 10^{15} FLOPs, the non-looped model gives the shorter estimated code; its model part reaches 1.45×10^7 bits.
- Above that budget, the looped model gives the shorter code. Repeated application of the simple dynamics becomes preferable.
- The authors interpret this as emergence: limited compute favours prediction from large-scale patterns.

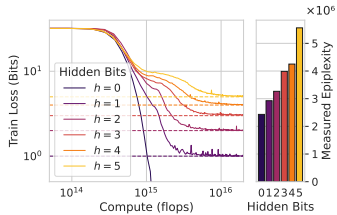


Two-part code and S_T against compute in FLOPs. Source: Fig. 6 of Finzi et al. (2026).

Paradox 3 in the experiment: hidden inputs

Setting. Sample $Z \sim U_{32}$; hide $h = 0, \ldots, 5$ bits to give $X = m(Z)$; predict $Y = f(Z)$, where f applies four rule 30 updates. Transformer; requential code.





Left: the data. Right: training loss against compute in FLOPs (dashed: h bits) and S_T for each h. Source: Fig. 5 of Finzi et al. (2026).

$$Q(y \mid x) = 2^{-h} \sum_{z:\, m(z)=x} \mathbf{1}\{f(z) = y\}.$$

Generating a pair uses one complete state; direct probability evaluation enumerates 2^h completions. As h increases, the experiment requires more training compute and gives a larger model-part estimate.

Enumeration is one algorithm, not a complexity lower bound; the experiment does not identify the learned algorithm.

Experiment: the order of chess data

Setting. Lichess games in two orders: moves → final board, or board → moves. Transformer sweep with requential coding.

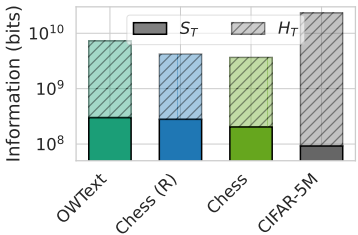


Left: H_T and S_T against compute in FLOPs. Right: S_T and accuracy after fine-tuning. Source: Figs. 4c and 7 of Finzi et al. (2026).

- At large compute, reverse order gives higher H_T and S_T.
- After fine-tuning, reverse-order models reach 0.28 board-evaluation accuracy versus 0.21 for forward-order models; puzzle accuracy is similar.

Application: structure in natural data

Setting. Equal token budgets and a shared compute bound. Compare OpenWebText, chess and CIFAR-5M with the requential code.



Chess (R): reverse order. Source: Fig. 8a of Finzi et al. (2026). Table: our estimates from the figure.

Dataset	S_T, bits	MDL_T, bits	S_T/MDL_T
OpenWebText	3×10^8	7×10^9	4%
Chess, reverse	2.8×10^8	4×10^9	7%
Chess, forward	2×10^8	3.5×10^9	6%
CIFAR-5M	9×10^7	2.3×10^{10}	0.4%

- These measurements compare representations at a fixed token count, rather than equal bytes or semantic content.
- The estimates depend on the tokenisation, model family and compute budget.

Conclusions and limitations

Summary

- S_T counts selected model bits; H_T is the expected data-code length.
- Computational restrictions can change both the total description length and its decomposition.
- Neural-network estimates combine a model-code estimate with validation loss, optimized under a compute budget.

Limitations

- S_T is not a guarantee of downstream usefulness.
- The existence theorem is conditional on cryptographic assumptions and proves $\Omega(\log_2 n)$ growth.
- Estimates depend on architecture, optimiser, T, dataset size and random seed.

Follow-up work from Wilson's group

Qiu et al. (2026): **Requential Coding**

A teacher selects examples drawn from the student's predictive distribution. Recording the selection indices reconstructs the student's training trajectory; the coding cost depends on teacher–student disagreement. This yields PAC–Bayes generalization bounds for billion-parameter language models. At fixed loss, larger models can have shorter codes.

Su et al. (2026): **EpiSelect and EpiGen**

EpiSelect fits cross-domain loss-scaling laws and adjusts sampling weights to maximize predicted gains in an online epiplexity proxy. **EpiGen** trains a synthetic-data generator with a policy-gradient reward: the reduction in learner loss on earlier samples after training on a new batch. Both methods report improved downstream performance.

Requential coding constructs a model code. EpiSelect and EpiGen optimize online proxies; their link to the formal S_T remains open. Titles link to the papers.

Literature: epiplexity and follow-up work

Marc Finzi, Shikai Qiu, Yiding Jiang, Pavel Izmailov, J. Zico Kolter, Andrew Gordon Wilson (2026).
From Entropy to Epiplexity.

Shikai Qiu, Marc Finzi, Yujia Zheng, Kun Zhang, Andrew Gordon Wilson (2026).
Requential Coding.

Ellen Su, Andres Potapczynski, Shikai Qiu, Edward Hughes, Andrew Gordon Wilson (2026).
Epiplexity Guided Data Selection and Generation for Out-of-Distribution Generalization.

Literature: background

Ming Li and Paul Vitányi (2008).
An Introduction to Kolmogorov Complexity and Its Applications.

Stephen Wolfram (2002).
A New Kind of Science.

Yiding Jiang, Allan Zhou, Zhili Feng, Sadhika Malladi, J. Zico Kolter (ICLR, 2025).
Adaptive Data Optimization: Dynamic Sample Selection with Scaling Laws.

Questions for the audience: answers

1. Let $X \sim U_n$. Which quantity is close to n: $S_T(X)$ or $H_T(X)$?

Answer

$H_T(X) = n + O(1)$. A constant-length uniform model gives $\mathrm{MDL}_T(X) \leq n + c$; since $H_T(X) \geq n$, the selected model has $S_T(X) \leq c$.

2. In the prequential estimate, which part of the training-loss curve approximates the epiplexity?

Answer

The cumulative excess loss above the final-model baseline: $\sum_i [-\log_2 P_i(Z_i) + \log_2 P_M(Z_i)]$.

3. If $P = Q$, must the model use the same algorithm as the generator?

Answer

No. Equality fixes probabilities. A generator can sample one hidden state, while likelihood evaluation may require summing over many possible states.

Backup: basic properties

Proposition 1 (Sec. 3 of the paper)

Let c_1, c_2 be constants independent of X. For a random variable X on $\{0,1\}^n$:

1. $S_T(X) \geq 0$ and $H_T(X) \geq 0$;
2. $H(X) \leq \mathrm{MDL}_T(X) \leq n + c_1$;
3. $\mathrm{MDL}_{T'}(X) \leq \mathrm{MDL}_T(X)$ whenever $T'(n) \geq T(n)$;
4. $\mathrm{MDL}_{T'}(f^{-1}(X)) \leq \mathrm{MDL}_T(X) + |\mathrm{f}| + c_2$ with $T'(n) = T(n) + \mathrm{Time(f)}$, for a program f that computes a bijection f in fixed time.

Property 4 is the analogue of $K(f(x)) \leq K(x) + K(f) + c$. Under a time bound a short program for f^{-1} gives no short program for f.

Theorem 1 (Thm. 10 of the paper)

If one-way functions secure against probabilistic polynomial-time algorithms with polynomial advice exist, then some random variables X_n on $\{0,1\}^n$ have $S_{\mathrm{Poly}}(X_n) = \Omega(\log_2 n)$.